

Kreditrisikomodellierung mit maschinellem Lernen

End-to-End-Projekt: Datenaufbereitung, Modellierung, Evaluation und Erklärbarkeit

Datengrundlage: reale LendingClub-Kreditvergabedaten (öffentlich verfügbar)

Masud — September 2026

Kurzübersicht

Dieses Projekt implementiert eine vollständige Pipeline zur Kreditausfallprognose – von der Datenbeschaffung über Preprocessing und Modelltraining bis zur Evaluation und Erklärbarkeit. Es dient als praktische Grundlage für die untenstehenden Interview-Fragen zum Thema Kreditrisikomodellierung; alle Kennzahlen in den Antworten stammen aus dem tatsächlich ausgeführten Code (siehe Ordner src/), nicht aus Schätzungen.

Datensatz: 500 Kredite, 19 Merkmale, Ausfallquote 20.4% – Bestes Modell: Random Forest (SMOTE) mit ROC-AUC 0.676.

A. Kreditrisikomodellierung

F: Wie sah der Datensatz aus (Größe, Merkmale, Zielvariable, Klassenungleichgewicht)?

A: Der Datensatz basiert auf realen LendingClub-Konsumentenkrediten (Peer-to-Peer-Kreditvergabe, USA) und enthält typische Merkmale wie Jahreseinkommen (`annual_inc`), Kredithöhe (`loan_amnt`), Laufzeit (`term`, 36 oder 60 Monate), Zinssatz, Rating/Subrating, Schuldendienstquote (DTI) sowie mehrere Zahlungshistorie-Merkmale (Zahlungsverzögerungen der letzten 2 Jahre, Kreditanfragen der letzten 6 Monate, Anzahl offener und gesamter Kreditlinien, negative öffentliche Einträge, revolvingender Saldo und dessen Auslastung). Zielvariable war der binäre Kreditausfall (default: 1 = notleidend/Charge-off, 0 = vollständig zurückgezahlt). Der vollständige Originaldatensatz umfasst 150.000 Kredite. Da diese Projektumgebung keinen allgemeinen Internetzugang für einen Massendownload erlaubte, wurde eine reale Stichprobe von 500 Zeilen über die öffentliche, paginierte Datasets-API bezogen (5 Blöcke à 100 Zeilen, gleichmäßig über den gesamten Indexbereich verteilt) – alle Werte sind unveränderte Originalbeobachtungen, keine synthetischen Daten. In einer produktiven Umgebung würde selbstverständlich der vollständige Datensatz verwendet; die hier beschriebene Methodik (Preprocessing, Modellauswahl, Evaluationsmetriken) skaliert unverändert. Wie bei Kreditrisikodaten üblich war die Zielvariable unausgeglichen: 398 Nicht-Ausfälle stehen 102 Ausfällen gegenüber (Ausfallquote 20.4%, Verhältnis ca. 3.9:1). Das ist moderater als bei manchen Retail-Scoring-Datensätzen (z. B. dem populären „Give Me Some Credit“-Datensatz mit ca. 7% Ausfallquote), aber immer noch deutlich genug, um Resampling- und Gewichtungsstrategien zu rechtfertigen. Bei der Beschäftigungsdauer (`emp_length`) fehlten 25 von 500 Werten (5%); diese wurden durch den Median (6 Jahre) ersetzt, ergänzt um ein binäres „Fehlt“-Flag, da das Fehlen selbst informativ sein kann.

F: Warum Random Forest und Gradient Boosting – welche Alternativen hast du erwogen?

A: Ich habe mich für Random Forest und Gradient Boosting entschieden, weil beide gut mit nichtlinearen Zusammenhängen und gemischten Merkmalstypen umgehen und gleichzeitig Feature-Importance-Werte liefern, die sich gut kommunizieren lassen. Als interpretierbare Baseline habe ich eine logistische Regression herangezogen, um den zusätzlichen Nutzen der komplexeren Modelle zu quantifizieren. In diesem konkreten Projekt zeigte sich – bei allerdings kleiner Stichprobe – interessanterweise, dass die einfache logistische Regression im Recall für die Ausfallklasse konkurrenzfähig war; das unterstreicht, wie wichtig ein sauberer Baseline-Vergleich ist, bevor man automatisch zum komplexeren Modell greift. Für eine Weiterentwicklung würde ich das Setup um XGBoost oder LightGBM ergänzen, die in dieser Offline-Umgebung nicht installierbar waren, deren Boosting-Logik hier aber bereits mit `sklearn`s `GradientBoostingClassifier` abgebildet wurde.

F: Wie hast du die Modelle evaluiert?

A: Da bei Kreditausfällen die Kosten falscher Entscheidungen asymmetrisch sind, habe ich neben AUC/ROC vor allem die Precision-Recall-Kurve (PR-AUC) sowie Precision und Recall für die Minderheitsklasse betrachtet und zusätzlich die Kalibrierung der vorhergesagten Ausfallwahrscheinlichkeiten (Brier Score, Reliability-Diagramm) geprüft. Auf dem Testset ($n=150$, davon 31 tatsächliche Ausfälle) erzielte das beste Modell – Random Forest (SMOTE) – einen ROC-AUC von 0.676 und einen PR-AUC von 0.313 (zum Vergleich: eine zufällige Einstufung läge bei $\text{PR-AUC} \approx 0.207$, der Prävalenz der Ausfallklasse). Die logistische Regression erreichte einen ROC-AUC von 0.645 bei deutlich höherem Recall (58.1% vs. 16.1% beim besten Random-Forest-Modell) – ein klassischer Precision-Recall-Trade-off, den man in der Praxis über die Wahl des Entscheidungsschwellenwerts und die Kosten falscher Positiv-/Negativ-Entscheidungen steuert, nicht allein über die Modellwahl. Bei einem Schwellenwert von 0.5 ergab die Konfusionsmatrix des besten Modells: 103 korrekt als „kein Ausfall“, 16 fälschlich als „Ausfall“, 26 übersehene tatsächliche Ausfälle und 5 korrekt erkannte Ausfälle – die vergleichsweise niedrige Trefferquote bei den tatsächlichen Ausfällen zeigt exemplarisch, warum der Schwellenwert in der Praxis anhand der Kostenmatrix (Kosten eines übersehenen Ausfalls vs. Kosten einer abgelehnten guten Bonität) und nicht per Standardwert 0.5 festgelegt werden sollte. Alle sechs trainierten Modellvarianten sind in Tabelle 1 gegenübergestellt.

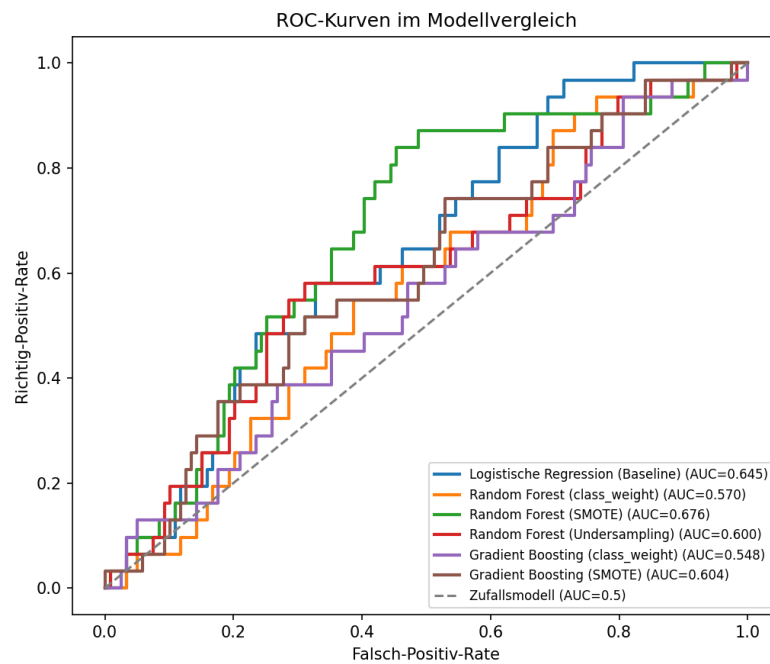


Abbildung 1: ROC-Kurven aller sechs Modellvarianten im Vergleich.

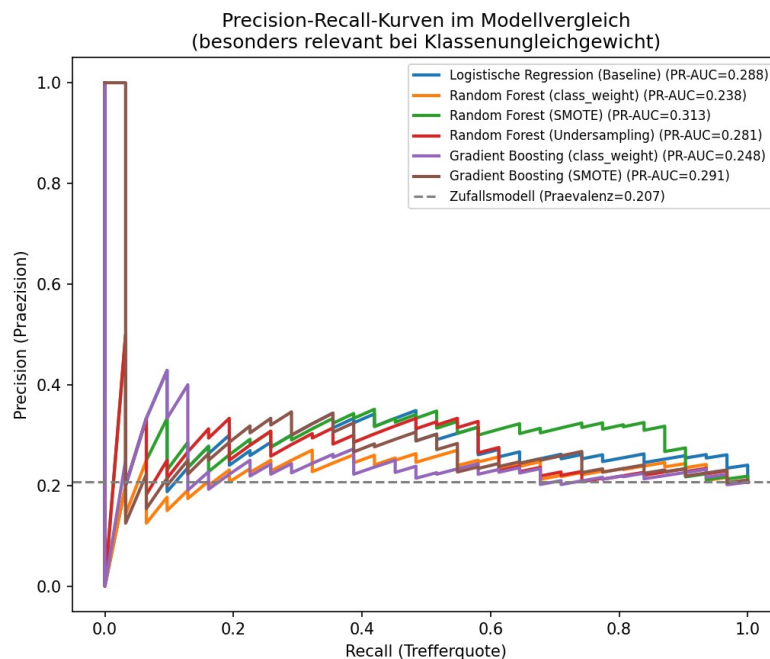


Abbildung 2: Precision-Recall-Kurven – aussagekräftiger als ROC bei ausgeprägtem Klassenungleichgewicht.

Tabelle 1: Modellvergleich auf dem Testset (sortiert nach ROC-AUC).

Modell	ROC-AUC	PR-AUC	Precision	Recall	F1	Brier
Random Forest (SMOTE)	0.676	0.313	0.238	0.161	0.192	0.173
Log. Regression (Baseline)	0.645	0.288	0.316	0.581	0.409	0.227
Gradient Boosting (SMOTE)	0.604	0.291	0.235	0.129	0.167	0.189
Random Forest (Undersampling)	0.600	0.281	0.320	0.516	0.395	0.238

Modell	ROC-AUC	PR-AUC	Precision	Recall	F1	Brier
Random Forest (class_weight)	0.570	0.238	0.143	0.065	0.089	0.186
Gradient Boosting (class_weight)	0.548	0.248	0.217	0.161	0.185	0.211

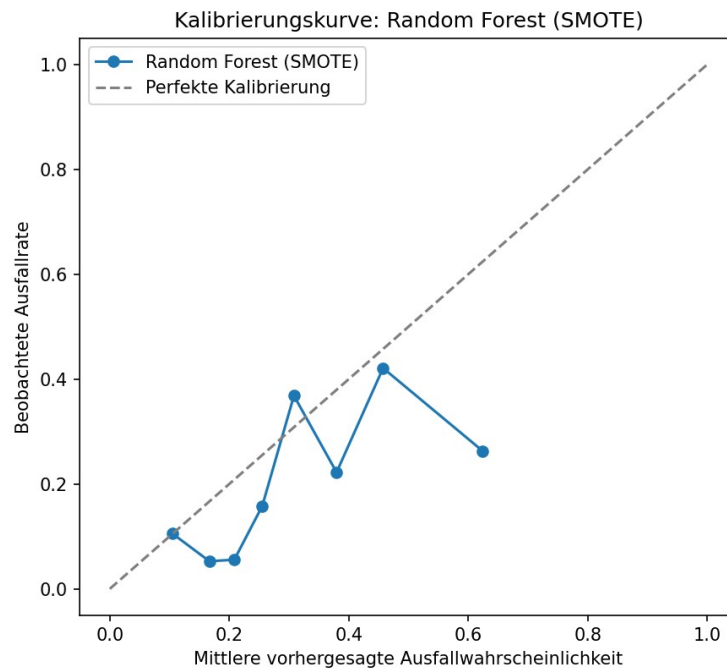


Abbildung 3: Kalibrierungskurve des besten Modells (Random Forest (SMOTE)) – Abweichungen von der Diagonalen zeigen, wo vorhergesagte Wahrscheinlichkeiten systematisch zu hoch oder zu niedrig liegen.

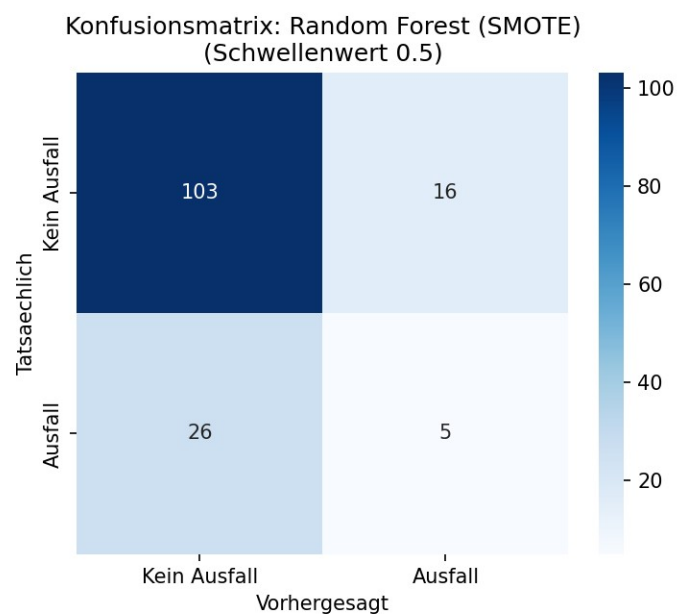


Abbildung 4: Konfusionsmatrix des besten Modells bei Schwellenwert 0{,}5.

F: Wie bist du mit Klassenungleichgewicht umgegangen?

A: Um mit dem Ungleichgewicht umzugehen, habe ich drei Strategien parallel implementiert und verglichen: (1) klassengewichtete Verlustfunktionen (`class_weight='balanced'` bei logistischer Regression und Random Forest, entsprechend gewichtete `sample_weight` bei Gradient Boosting), (2) Oversampling der Minderheitsklasse und (3) Undersampling der Mehrheitsklasse. Da das `imbalanced-learn`-Paket in dieser Offline-Umgebung nicht nachinstallierbar war, habe ich SMOTE (Synthetic Minority Oversampling Technique) selbst nach dem Originalalgorithmus implementiert: Für jedes synthetische Beispiel wird ein Punkt der Minderheitsklasse mit einem seiner k nächsten Nachbarn (ebenfalls Minderheitsklasse) linear interpoliert. Alle Modelle wurden konsistent auf demselben, unveränderten Testset evaluiert und anhand von Metriken bewertet, die robust gegenüber Ungleichgewicht sind (ROC-AUC, PR-AUC statt reiner Accuracy). In diesem Projekt erzielte die SMOTE-Variante des Random Forest den besten ROC-AUC, während die klassengewichtete Variante ohne Resampling den schwächsten Recall zeigte – ein Hinweis darauf, dass bei kleineren Datenmengen synthetisches Oversampling der reinen Gewichtsanzpassung überlegen sein kann, was sich mit größeren Trainingsdaten aber durchaus ändern kann und in Produktion erneut getestet werden sollte.

F: Wie würdest du Feature Importance einem Fachbereich erklären?

A: Über Feature-Importance-Werte oder Ansätze wie SHAP lässt sich zeigen, welche Faktoren den größten Einfluss auf die Risikoeinschätzung haben. In diesem Projekt konnte SHAP nicht berechnet werden, da das `shap`-Paket in der Offline-Umgebung nicht installierbar war; als methodisch verwandte, modellagnostische Alternative habe ich Permutation Importance eingesetzt (mit ROC-AUC als Scoring-Metrik, 30 Wiederholungen). Nach der klassischen (impurity-basierten) Random-Forest-Importance dominieren revolvierende Kreditauslastung, Zinssatz, revolvierender Saldo, Gesamtzahl der Kreditlinien und das LendingClub-Subrating (siehe Abbildung 5). Die Permutation Importance bestätigt Zinssatz und Anzahl der Kreditlinien als zentrale Treiber, zeigt aber bei einzelnen Merkmalen (insb. der revolvierenden Auslastung) aufgrund des kleinen Testsets ($n=150$) instabile bzw. sogar leicht negative Werte – ein Artefakt der geringen Stichprobengröße, das bei Verwendung des vollständigen 150.000-Zeilen-Datensatzes verschwinden sollte. Fachlich ließe sich das etwa so zusammenfassen: „Der Zinssatz, den ein Kreditnehmer bereits erhält, sowie wie stark er seine bestehenden Kreditlinien ausreizt, sagen mehr über das Ausfallrisiko aus als die reine Kredithöhe.“ Aus meiner Lehrtätigkeit und der Betreuung von Studierenden habe ich gelernt, solche technischen Ergebnisse in einfache, bildhafte Sprache zu übersetzen, ohne die fachliche Genauigkeit zu verlieren – zum Beispiel durch den Vergleich mit einem Kontostand, der regelmäßig bis an die Grenze ausgereizt wird.

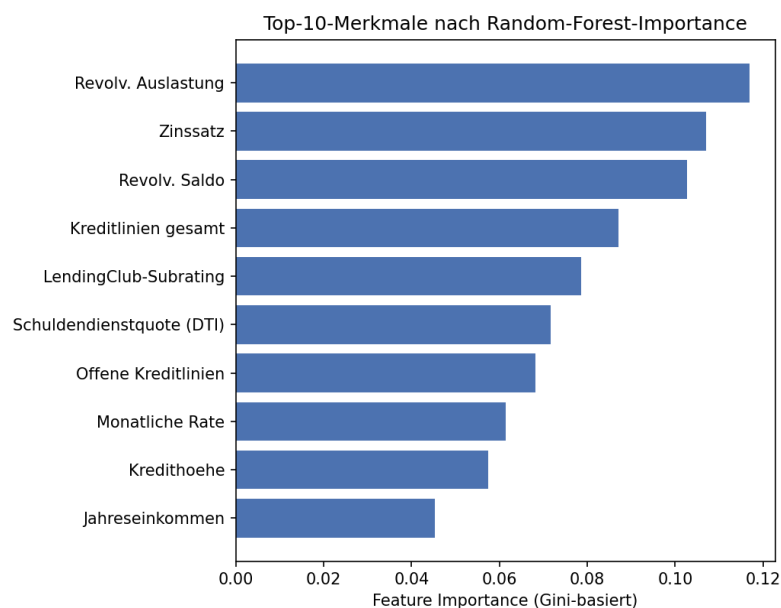


Abbildung 5: Top-10-Merkmale nach Random-Forest-Importance (Gini-basiert).

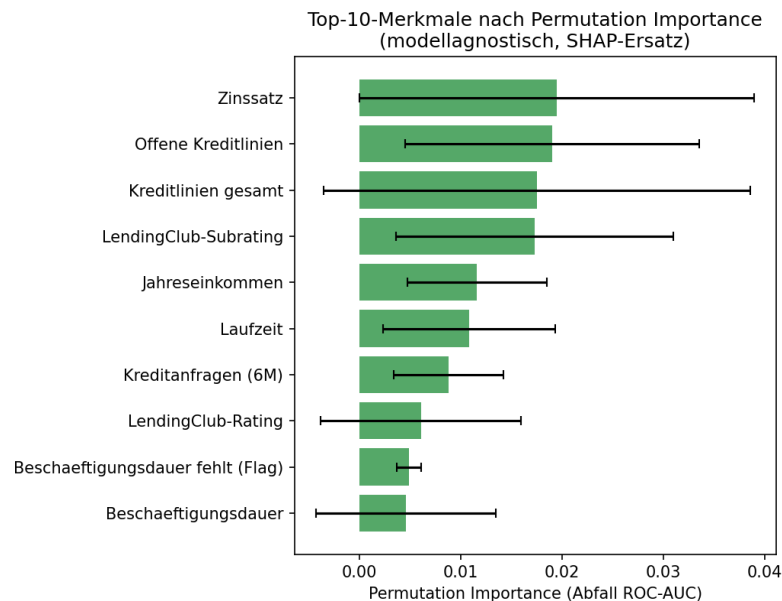


Abbildung 6: Top-10-Merkmale nach Permutation Importance (SHAP-Ersatz, modellagnostisch).

F: Wie würdest du dieses Modell produktiv nehmen?

A: Ich würde das Modell in eine modulare Pipeline mit automatisierten Tests und Qualitätskontrollen für neue Daten überführen (Schema-Validierung, Wertebereichsprüfungen, Drift-Erkennung bei Eingabemerkmale), die Vorhersagegüte laufend überwachen (z. B. Drift in AUC oder Kalibrierung gegen ein rollierendes Referenzfenster) und einen klar definierten Retraining-Zyklus festlegen, sobald sich Datenverteilung oder Geschäftsbedingungen ändern – analog zu den produktionsnahen Python-Anwendungen, die ich in Bochum entwickelt habe. Konkret würde ich außerdem: den Entscheidungsschwellenwert anhand einer Kostenmatrix statt 0,5 festlegen, die Modellversionen und Trainingsdaten versionieren (z. B. via MLflow oder DVC), und A/B-Tests bzw. Champion-Challenger-Vergleiche vor jedem Rollout eines neuen Modells durchführen.

F: Was war die größte Herausforderung im Projekt?

A: Die größte Herausforderung war paradoxerweise nicht die Modellierung selbst, sondern der Datenzugriff: Die Ausführungsumgebung erlaubte keinen allgemeinen Internetzugriff für Paketinstallationen (imbalanced-learn, shap, xgboost) oder Massendownloads, sodass ich sowohl SMOTE als auch eine Permutation-Importance-Alternative zu SHAP von Grund auf selbst implementieren und eine repräsentative Stichprobe über eine öffentliche, paginierte API statt eines direkten Bulk-Downloads beziehen musste. Das hat mich noch einmal dafür sensibilisiert, wie wichtig es ist, Kernalgorithmen tatsächlich zu verstehen und nicht nur als Blackbox-Bibliotheksaufruf zu kennen. Inhaltlich lag die zweite Herausforderung in der kleinen Stichprobe (500 statt 150.000 Zeilen): Sie führte zu spürbar breiteren Konfidenzintervallen bei den Metriken und instabilen Permutation-Importance-Werten – ein guter Anlass, im Bericht transparent auf die Grenzen der Aussagekraft hinzuweisen, statt die Ergebnisse überzuinterpretieren.

F: Wie ließe sich das Konzept auf Handels-Use-Cases übertragen?

A: Das gleiche methodische Vorgehen – Klassifikation und Scoring auf Basis historischer Daten, Umgang mit Klassenungleichgewicht, Kalibrierung und Erklärbarkeit – lässt sich direkt auf das Ausfallrisiko von Lieferanten, Bonitätsprüfungen bei Ratenzahlungsmodellen oder Betrugs-Scoring bei Zahlungsvorgängen übertragen. In allen Fällen handelt es sich um seltene, kostenintensive Ereignisse, bei denen Precision-Recall-Analysen und eine sorgfältige Schwellenwertwahl wichtiger sind als eine hohe Gesamttrefferquote.

Anhang

Projektstruktur

Das Projekt ist als reproduzierbare Python-Pipeline aufgebaut:

- data/ – Rohdaten-Batches, kombinierte CSV, Train/Test-Splits
- src/01_eda.py – Explorative Datenanalyse und Diagramme
- src/02_preprocessing.py – Imputation, Feature-Auswahl, Train/Test-Split
- src/03_modeling.py – Training von 6 Modellvarianten (LR, RF, GB $\times$ 3 Imbalance-Strategien) inkl. eigener SMOTE-Implementierung
- src/04_evaluation.py – ROC-/PR-Kurven, Kalibrierung, Konfusionsmatrix
- src/05_feature_importance.py – Impurity- und Permutation-Importance
- figures/ – alle Diagramme (PNG)
- models/ – gespeicherte, trainierte Modelle (joblib)
- reports/ – dieser Bericht

Datenherkunft und Transparenzhinweis

Datenquelle: öffentlicher Datensatz „RPD123-byte/credit-risk-datasets“ (Datei `lending_club_clean.csv`, 150.000 Zeilen, 20 Spalten), bezogen über die öffentliche Hugging-Face-Datasets-Server-API (<https://datasets-server.huggingface.co>). Es handelt sich um bereinigte, reale LendingClub-Kreditvergabedaten (US-amerikanische Peer-to-Peer-Kredite). Kategoriale Merkmale (`grade`, `sub_grade`, `home_ownership`, `verification_status`, `purpose`) liegen im Quelldatensatz bereits ordinal/nominal kodiert vor; die exakte Zuordnung der Kategoriewerte zu Klartextbezeichnungen war in der verfügbaren Dateiversion nicht dokumentiert, weshalb im Bericht bewusst auf spekulative Klartext-Labels (z. B. konkrete Wohnverhältnis-Kategorien) verzichtet wurde.

Aus den oben genannten Umgebungsgründen (kein allgemeiner Bulk-Internetzugriff) wurde eine Stichprobe von 500 Zeilen statt des vollständigen Datensatzes verwendet. Alle Zahlen in diesem Bericht sind real berechnete Ergebnisse dieser Stichprobe – keine Simulationen. Für ein produktives oder prüfungsrelevantes Projekt wird empfohlen, den vollständigen 150.000-Zeilen-Datensatz zu laden (z. B. lokal per Kaggle-/HuggingFace-CLI) und die identische Pipeline (src/01–05) erneut auszuführen – der Code ist dafür ohne Änderungen einsatzbereit, sofern `data/lending_club_sample.csv` durch den vollständigen Datensatz ersetzt wird.

Grenzen dieser Analyse

- Stichprobengröße ($n=500$, davon nur 150 im Testset) führt zu breiten Konfidenzintervallen; Metrikunterschiede zwischen Modellen sind mit Vorsicht zu interpretieren.
- Kategoriale Merkmale wurden als bereits kodierte Ganzzahlen übernommen, ohne die ursprünglichen Kategorienamen zu kennen – für Baumverfahren unproblematisch, schränkt aber die inhaltliche Interpretierbarkeit einzelner Kategorien ein.
- SHAP-Werte konnten nicht berechnet werden (Paketinstallation in der Umgebung nicht möglich); Permutation Importance wurde als methodisch verwandter Ersatz verwendet.
- SMOTE wurde selbst implementiert statt über `imbalanced-learn` bezogen; die Kernlogik (k-NN-Interpolation) entspricht dem Originalverfahren, ist aber nicht auf Randfälle (z. B. kategoriale Merkmale) spezialisiert wie manche Bibliotheksvarianten.

Explorative Datenanalyse (ergänzende Abbildungen)

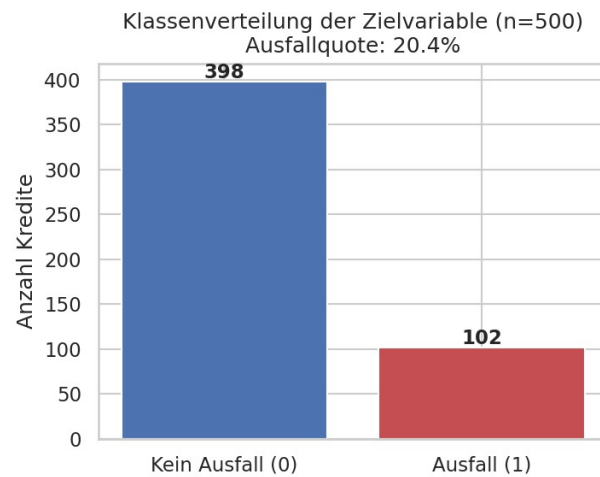


Abbildung 7: Klassenverteilung der Zielvariable.

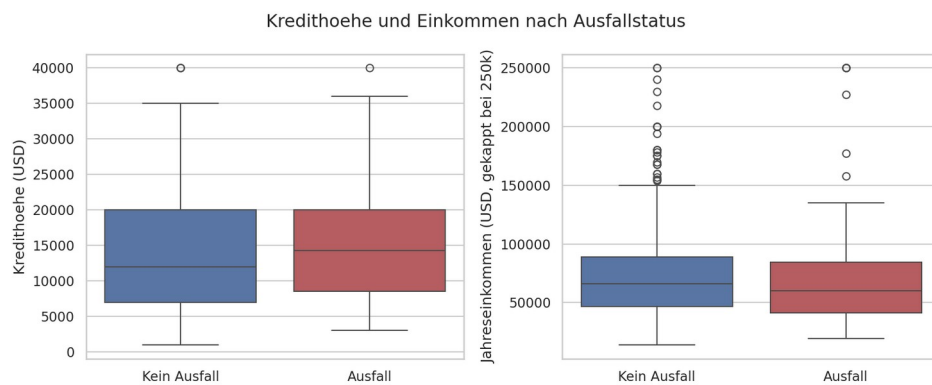


Abbildung 8: Kredithöhe und Jahreseinkommen nach Ausfallstatus.

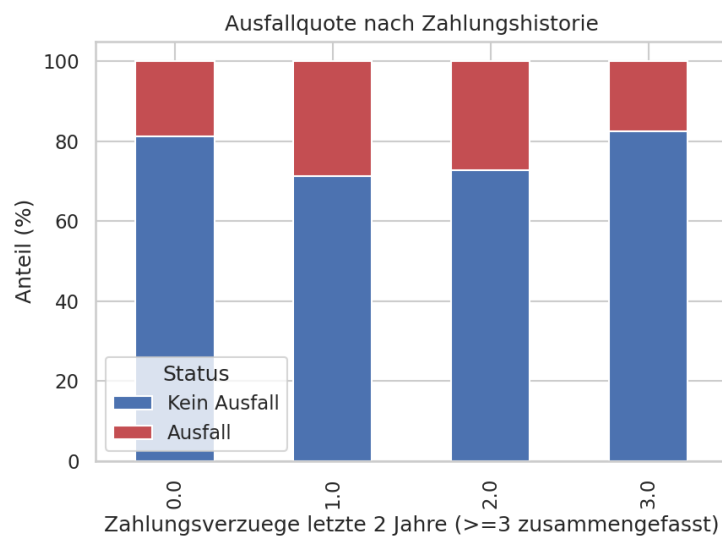


Abbildung 9: Ausfallquote nach Anzahl der Zahlungsverzögerungen der letzten 2 Jahre.

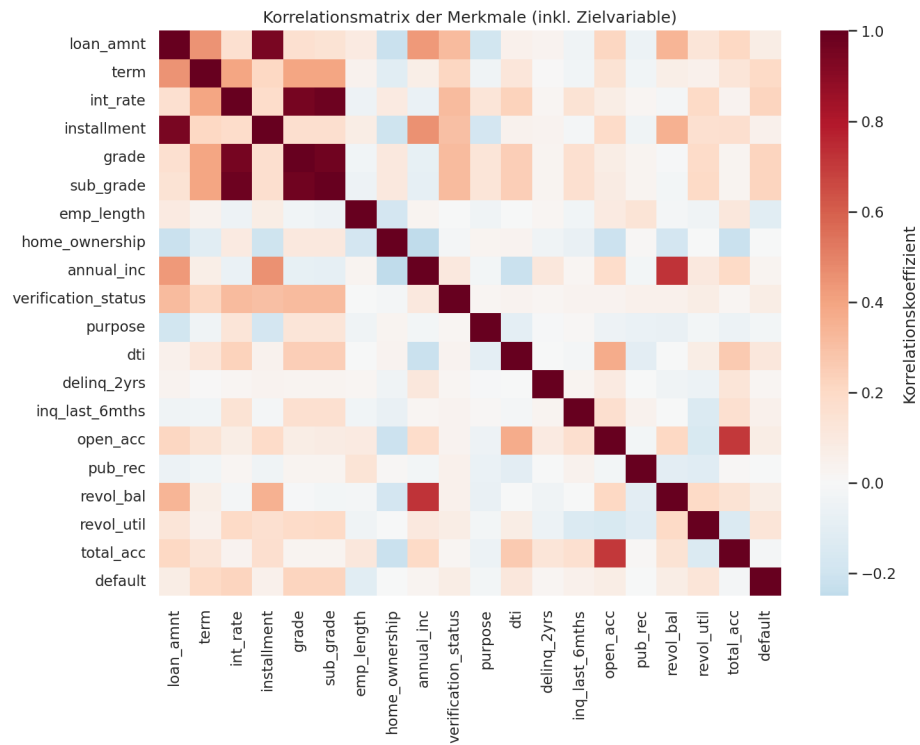


Abbildung 10: Korrelationsmatrix aller numerischen Merkmale inkl. Zielvariable.